Arabic Sentiment Analysis System

NLP Project Documentation

Tariq Aburajab : 2110025
Ahmad Abu-Khdair : 2110127

Abstract

This report documents the development and implementation of an Arabic sentiment analysis system that classifies Arabic text into negative, neutral, or positive sentiment categories. The system utilizes spaCy's natural language processing capabilities to build a text classification model specifically designed for Arabic language processing. Performance evaluation shows promising results with approximately 83% accuracy on test data.

1. Introduction

Sentiment analysis, the computational study of people's opinions and emotions expressed in text, is a rapidly growing field in Natural Language Processing (NLP). While substantial work exists for English language sentiment analysis, Arabic presents unique challenges due to its complex morphology, dialectal variations, and rich figurative expressions.

This project implements a machine learning-based sentiment analysis system for Arabic text, capable of classifying input text into three sentiment categories: negative, neutral, and positive. The system uses spaCy to create, train, and deploy a custom text categorization model trained on labeled Arabic text data.

2. Project Objectives

* Design and implement a sentiment analysis system for Arabic text
* Process and prepare Arabic text dataset for machine learning
* Develop a classification model with high accuracy and balanced performance
* Implement an interactive interface for testing user input
* Evaluate model performance using standard metrics

3. Data Description

The system uses labeled Arabic text data stored in a CSV file with columns for text content and sentiment labels. The dataset was preprocessed to handle:
* Missing values
* Duplicate entries
* Consistent label formatting (mapping text labels to numeric values)
* Class imbalance through weighted learning

An 80/20 train-test split was used for model development and evaluation, with stratification to maintain class distribution across splits.

4. Methodology

4.1 Data Preprocessing

* Text cleaning and normalization
* Label encoding: negative (0), neutral (1), positive (2)
* Duplicate detection and removal
* Statistical analysis of class distribution

4.2 Model Architecture

The system uses spaCy's text categorization pipeline with the following components:
* A blank Arabic language model
* A text categorizer component for classification
* Custom training data formatting for spaCy's Example objects
* Class weighting to handle imbalanced data

4.3 Training Process

The model training process involves:
* Converting text and labels to spaCy's expected format
* Creating Example objects for training
* Batching examples using minibatch with dynamic compounding
* Applying dropout (0.2) for regularization
* Evaluating on a subset of training data during epochs
* Saving the best model to disk

5. Implementation Details

5.1 System Components

* `main.py`: Handles data loading, preprocessing, training, evaluation, and interactive mode
* `sentiment.py`: Contains the `ArabicSentimentAnalyzer` class that implements model operations
* `models/`: Directory for storing trained models

5.2 Key Features

* Dynamic handling of input data format
* Class weight computation for balanced learning
* Comprehensive evaluation metrics
* Interactive prediction interface for user testing
* Model persistence for later reuse

6. Results and Evaluation

6.1 Performance Metrics

The model achieved the following performance on the test set:
* Overall Accuracy: 83%
* Class-specific metrics:
  * Negative: Precision 81%, Recall 86%, F1-score 83%
  * Positive: Precision 85%, Recall 79%, F1-score 82%
* Balanced performance across classes with macro-average F1-score of 83%

6.2 Error Analysis

Analysis of misclassifications reveals challenges with:
* Dialectal Arabic variations
* Sarcasm and figurative language
* Context-dependent expressions
* Short texts with limited sentiment indicators

7. Discussion

The developed system demonstrates strong performance in sentiment analysis for Arabic text. The balanced metrics across sentiment classes indicate that the model works effectively for both positive and negative sentiment detection without significant bias.

The spaCy-based approach provides advantages in terms of memory efficiency and deployment simplicity compared to larger transformer models, making it suitable for scenarios with limited computational resources.

Areas for potential improvement include:
* More sophisticated Arabic-specific text preprocessing
* Incorporation of dialectal features
* Larger and more diverse training datasets
* Hybrid approaches combining rule-based and machine learning methods

8. Conclusion

This project successfully implemented an Arabic sentiment analysis system with good performance metrics. The system demonstrates the effectiveness of spaCy for developing specialized NLP applications for languages beyond English. The interactive interface allows for easy testing and demonstration of the model's capabilities.

The developed system provides a solid foundation for Arabic text sentiment analysis that can be extended and improved in future work.

Appendix

A: Code Samples

Key implementation of sentiment analysis class:
```python
class ArabicSentimentAnalyzer:
    def __init__(self, model_dir='models', num_labels=3):
        """Initialize the analyzer with spaCy model for Arabic text"""
        self.model_dir = model_dir
        self.num_labels = num_labels
```

```
        # Create a blank Arabic spaCy model
        self.nlp = spacy.blank("ar")

        # Add text categorizer
        if "textcat" not in self.nlp.pipe_names:
            self.textcat = self.nlp.add_pipe("textcat")

            # Add sentiment categories
            self.textcat.add_label("NEGATIVE")
            self.textcat.add_label("NEUTRAL")
            self.textcat.add_label("POSITIVE")
```

Example prediction code:
```python
def predict(self, texts):
    """Make predictions for a list of texts"""
    predictions = []

    for text in texts:
        doc = self.nlp(text)
        scores = doc.cats

        # Convert scores to labels
        pred_label = -1
        confidence = 0

        if scores["NEGATIVE"] >= scores["NEUTRAL"] and scores["NEGATIVE"] >=
scores["POSITIVE"]:
            pred_label = 0
            confidence = scores["NEGATIVE"]
        elif scores["NEUTRAL"] >= scores["NEGATIVE"] and scores["NEUTRAL"] >=
scores["POSITIVE"]:
            pred_label = 1
            confidence = scores["NEUTRAL"]
        else:
            pred_label = 2
            confidence = scores["POSITIVE"]

        predictions.append((pred_label, confidence))

    return predictions
```

```

## Appendix B: Interactive System Usage

The system provides an interactive mode for testing:
```
=== Interactive Mode ===
Enter Arabic text to analyze sentiment (type 'exit' to quit):

Text: هذا الفيلم جميل جداً
Sentiment: Positive (Confidence: 0.89)

Text: الخدمة سيئة للغاية
Sentiment: Negative (Confidence: 0.92)

Text: exit
Analysis complete!
```
```